Código: Análise de Treliças Planas

```
import math
import time
import matplotlib.pyplot as plt

# ============================================================
# SISTEMA DE ANÁLISE DE TRELIÇAS PLANAS ISOSTÁTICAS
# Implementação manual do método de Gauss
# ============================================================


# ============================================================
# ELIMINAÇÃO DE GAUSS (MANUAL)
# ============================================================
def gauss_elimination(A, b):
 n = len(b)

 # Matriz aumentada
 M = [A[i][:] + [b[i]] for i in range(n)]

 # Eliminação
 for k in range(n):

 # Pivotamento parcial
 max_row = max(range(k, n), key=lambda i: abs(M[i][k]))  M[k],
M[max_row] = M[max_row], M[k]

 pivot = M[k][k]

 if abs(pivot) < 1e-12:
            raise ValueError("Sistema singular ou mal condicionado.")

 # Normalização da linha pivô
 for j in range(k, n + 1):
 M[k][j] /= pivot

 # Eliminação abaixo
 for i in range(k + 1, n):
 factor = M[i][k]
 for j in range(k, n + 1):
 M[i][j] -= factor * M[k][j]

 # Retrossubstituição
 x = [0] * n

 for i in range(n - 1, -1, -1):
 x[i] = M[i][n]

 for j in range(i + 1, n):
 x[i] -= M[i][j] * x[j]

 return x


# ============================================================
# MONTAGEM DA MATRIZ DE EQUILÍBRIO
```

```
# ============================================================
def build_equilibrium_matrix(nodes, bars, supports):  N =
len(nodes)
 B = len(bars)

 reactions = []

 for i, support in enumerate(supports):

 if support == "fixed":
 reactions.append((i, 'x'))
 reactions.append((i, 'y'))

 elif support == "roller":
 reactions.append((i, 'y'))

 unknowns = B + len(reactions)

 A = [[0.0 for _ in range(unknowns)] for _ in range(2 * N)]

 # Barras
 for b, (i, j) in enumerate(bars):

 xi, yi = nodes[i]
 xj, yj = nodes[j]

 dx = xj - xi
 dy = yj - yi

 L = math.sqrt(dx**2 + dy**2)

 cx = dx / L
 cy = dy / L

 # Nó i
 A[2 * i][b] = cx
 A[2 * i + 1][b] = cy

 # Nó j
 A[2 * j][b] = -cx
 A[2 * j + 1][b] = -cy

 # Reações
 for r, (node, direction) in enumerate(reactions):

col = B + r

 if direction == 'x':
 A[2 * node][col] = 1.0

 elif direction == 'y':
 A[2 * node + 1][col] = 1.0

 return A, reactions


# ============================================================
# RESOLVER TRELIÇA
# ============================================================
def solve_truss(nodes, bars, supports, loads):
```

```
 N = len(nodes)
 B = len(bars)

 A, reactions = build_equilibrium_matrix(nodes, bars, supports)

 # Vetor de forças externas
 b = []

 for i in range(N):
 Fx, Fy = loads[i]

 b.append(-Fx)
 b.append(-Fy)

 # Resolver sistema
 solution = gauss_elimination(A, b)

 bar_forces = solution[:B]
 reaction_forces = solution[B:]

 return bar_forces, reaction_forces, reactions


# ============================================================
# ESTATÍSTICAS
# ============================================================
def compute_statistics(bar_forces):
 max_force = max(bar_forces, key=lambda x: abs(x))  avg_force =
sum(abs(f) for f in bar_forces) / len(bar_forces)

 return {
 "max_force": max_force,
 "avg_force": avg_force
 }


# ============================================================
# DESENHAR TRELIÇA
# ============================================================
def plot_truss(nodes, bars, bar_forces):

 plt.figure(figsize=(10, 6))

 max_abs = max(abs(f) for f in bar_forces)

 for idx, (i, j) in enumerate(bars):

 xi, yi = nodes[i]
 xj, yj = nodes[j]

 force = bar_forces[idx]

 # Cor
 if force > 0:
 color = (0, 0, min(abs(force) / max_abs, 1))  else:
 color = (min(abs(force) / max_abs, 1), 0, 0)

 plt.plot([xi, xj], [yi, yj],
 color=color,
 linewidth=3)
```

```
 # Texto da barra
 mx = (xi + xj) / 2
 my = (yi + yj) / 2

 plt.text(mx, my,
 f"{force:.1f}",
 fontsize=8)

 # Nós
 for i, (x, y) in enumerate(nodes):
 plt.scatter(x, y, color='black')
 plt.text(x + 0.1, y + 0.1, f"N{i}")

 plt.title("Treliça - Azul = Tração | Vermelho = Compressão")
plt.axis('equal')
 plt.grid(True)
 plt.show()


# ============================================================
# GRÁFICO DE BARRAS
# ============================================================
def plot_bar_chart(bar_forces):

 plt.figure(figsize=(10, 5))

 labels = [f"B{i}" for i in range(len(bar_forces))]  colors =

['blue' if f > 0 else 'red' for f in bar_forces]

plt.bar(labels, bar_forces, color=colors)

 plt.title("Forças nas Barras")
 plt.ylabel("Força (N)")
 plt.grid(True)

 plt.show()


# ============================================================
# HISTOGRAMA
# ============================================================
def plot_histogram(bar_forces):

 plt.figure(figsize=(8, 5))

 plt.hist(bar_forces, bins=10)

 plt.title("Distribuição das Forças Internas")
plt.xlabel("Força (N)")
 plt.ylabel("Frequência")

 plt.grid(True)

 plt.show()


# ============================================================
# TRELIÇA TESTE PADRONIZADA
```

```
# ============================================================
def standard_truss():

 # Nós
 nodes = [
 (0, 0), # Nó 0
 (2, 2), # Nó 1
 (4, 0), # Nó 2
 (6, 2) # Nó 3
 ]

 # Barras
 bars = [
 (0, 1),
 (1, 2),
 (2, 3),
 (0, 2),
 (1, 3)
 ]

 # Apoios
 supports = [
 "fixed", # Nó 0
 "free", # Nó 1
 "free", # Nó 2
 "roller" # Nó 3
 ]

 # Cargas
 loads = [
 (0, 0),
 (0, 0),
 (0, -1000),
 (0, 0)
 ]

 return nodes, bars, supports, loads


# ============================================================
# MAIN
# ============================================================
def main():

 print("=" * 60)
 print("ANÁLISE DE TRELIÇA PLANA")
 print("=" * 60)

 nodes, bars, supports, loads = standard_truss()

start = time.time()

 bar_forces, reaction_forces, reactions = solve_truss(
nodes,
 bars,
 supports,
 loads
 )
 end = time.time()
```

```
 # ====================================================  #
RESULTADOS
 # ====================================================
print("\nFORÇAS NAS BARRAS:")

 for i, f in enumerate(bar_forces):

 if abs(f) < 1e-6:
 status = "NULA"

 elif f > 0:
 status = "TRAÇÃO"

 else:
 status = "COMPRESSÃO"

 print(f"Barra {i}: {f:.2f} N -> {status}")

print("\nREAÇÕES DE APOIO:")

 for i, r in enumerate(reaction_forces):

 node, direction = reactions[i]

 print(f"Nó {node} ({direction}): {r:.2f} N")  stats

= compute_statistics(bar_forces)

 print("\nESTATÍSTICAS:")

 print(f"Força média: {stats['avg_force']:.2f} N")
print(f"Barra mais solicitada: {stats['max_force']:.2f} N")

 print(f"\nTempo de execução: {end - start:.6f} s")

 # ====================================================  #
GRÁFICOS
 # ====================================================
plot_truss(nodes, bars, bar_forces)

 plot_bar_chart(bar_forces)

 plot_histogram(bar_forces)


# ============================================================
# EXECUÇÃO
# ============================================================
if __name__ == "__main__":
 main()
```